

Escola Superior de Tecnologia
Instituto Politécnico de Castelo Branco
Licenciatura em Engenharia Informática

Computação em Nuvem
Mini Projeto --- Relatório Final

OnzePerfeito
Plataforma de Fantasy Football — Liga Portugal

Elementos do Grupo:

- João Pedro de Sousa Pereira nº20221689
- Kyrylo Sakhnenko nº 20221710
- Rodrigo Coimbra Figueiredo nº 20221713

Docente: Osvaldo Arede dos Santos Curso:
Licenciatura em Engenharia Informática
Ano Letivo: 2025/2026

Índice

1. Introdução	5
2. Descrição do Projeto OnzePerfeito	6
2.1. Mecânicas Principais	6
2.2. Sistema de Ligas	6
2.3. Perfil do Utilizador e Base de Dados	6
3. Serviços Microsoft Azure Utilizados	7
4. Arquitetura da Solução	8
5. Planificação e Divisão de Tarefas	9
5.1. Divisão de Responsabilidades	9
6. Infraestrutura Base no Azure	10
6.1. Resource Group e Azure Cosmos DB	10
6.2. Azure BLOB Storage	10
7. GitHub e Azure App Service — CI/CD	11
7.1. Azure App Service	11
8. Backend Flask e Docker	12
8.1. Arquitetura e Rotas REST	12
8.2. Containerização com Docker e Azure Container Registry	12
9. Azure Functions	13
9.1. GerarPontuacoes	13
9.2. AtualizarPrecos	13
9.3. EventosEspeciais — Funcionalidade Inovadora	13
10. Script de Seed dos Jogadores	14
11. Template Bicep — Infrastructure as Code	15
12. Frontend — Desenvolvimento da Interface	16
12.1. Arquitetura e Ficheiros de Base	16
12.2. Módulos auth.js e api.js	16
12.3. Páginas HTML Desenvolvidas	17
13. Integração BLOB Storage e Deploy em Produção	19
13.1. Logos dos Clubes e Configuração de CORS	19
13.2. Fotos dos Jogadores — Wikipedia API com Fallback	19
13.3. Problemas de Deploy e Soluções	19
13.4. Arquitetura Final em Produção	20
14. Melhorias — Fase Final	22
14.1. Vista Tática com Formações — equipa.html	22
14.2. Módulo de Ligas — ligas.html	23

15. Testes de Integração	25
15.1. Resultados dos Testes	25
15.2. Teste End-to-End — Fluxo Completo	25
16. Estudo de Custos e Alternativas Cloud	27
16.1. Estimativa de Custos Mensais em Produção	27
16.2. Alternativas Cloud Europeias	28
17. Conclusão	30
18. Referências	31

1. Introdução

O presente relatório consolida toda a documentação produzida ao longo do semestre no âmbito do mini projeto de **Computação em Nuvem**, da Licenciatura em Engenharia Informática da Escola Superior de Tecnologia do Instituto Politécnico de Castelo Branco.

O projeto, denominado **OnzePerfeito**, consiste no desenvolvimento de uma plataforma web de *fantasy football* baseada na **Liga Portugal Betclic**, desenvolvida integralmente sobre a infraestrutura de cloud computing **Microsoft Azure**. A escolha do fantasy football como tema justifica-se pela popularidade deste tipo de aplicações — com exemplos reconhecidos como o Fantasy Premier League, o BIWENGER ou o Maisfutebol Fantasy — e pela sua adequação natural aos serviços Azure previstos no enunciado.

Este relatório está organizado de acordo com a progressão do projeto ao longo do semestre, cobrindo desde a conceção inicial da ideia (Avaliação #1), passando pelo estado intermédio de implementação (Avaliação #2), o desenvolvimento do frontend, a integração do BLOB Storage, o deploy em produção, as melhorias implementadas na fase final, os testes de integração e o estudo de custos.

É importante referir que o código JavaScript das Azure Functions e o template Bicep foram desenvolvidos com o auxílio da ferramenta de Inteligência Artificial Claude, conforme permitido pelo enunciado do projeto. Esta utilização é devidamente declarada ao longo do relatório.

2. Descrição do Projeto OnzePerfeito

O **OnzePerfeito** é uma aplicação web que permite aos utilizadores registados criar e gerir equipas virtuais de futebol, compostas por jogadores reais da Liga Portugal. Ao longo da época, os utilizadores acumulam pontos com base no desempenho simulado dos seus jogadores a cada jornada, competindo entre si em ligas globais ou em ligas privadas criadas entre amigos.

2.1. Mecânicas Principais

Cada utilizador dispõe de um saldo inicial de transferências, utilizado para seleccionar **15 jogadores** — 11 titulares e 4 suplentes — respeitando o limite orçamental disponível. Os jogadores têm **preços dinâmicos**: o preço de cada jogador é ajustado automaticamente com base na média das suas pontuações nas últimas jornadas, subindo quando o desempenho melhora e descendo quando piora.

As pontuações por jornada são geradas automaticamente por intervalos definidos por posição:

Posição	Intervalo de Pontuação
Guarda-redes	1 a 8 pontos por jornada
Defesa	1 a 10 pontos por jornada
Médio	1 a 12 pontos por jornada
Avançado	1 a 15 pontos por jornada

2.2. Sistema de Ligas

A plataforma suporta dois tipos de ligas: a **liga global**, onde todos os utilizadores registados competem entre si, e as **ligas privadas**, criadas por utilizadores que partilham um código de convite de 6 caracteres com os seus amigos. Em ambos os casos, existe um ranking atualizado com base nas pontuações acumuladas de cada equipa.

2.3. Perfil do Utilizador e Base de Dados

Cada utilizador tem acesso a um perfil pessoal com histórico de pontuações por jornada e classificação atual em cada liga. A base de dados de jogadores da Liga Portugal é criada manualmente e carregada através de um script de inicialização (*seed*). Cada jogador tem os seguintes atributos: nome completo, clube, posição, preço inicial e média de pontuações.

3. Serviços Microsoft Azure Utilizados

O OnzePerfeito utiliza o ecossistema Microsoft Azure da seguinte forma:

Serviço Azure	Utilização no Projeto
Azure App Service	Alojamento da aplicação web com deploy automático via GitHub Actions. Corre o container Docker com o backend Flask (Python) e o frontend HTML/CSS/JS.
GitHub + CI/CD	Repositório público com pipeline de deploy automático via GitHub Actions a cada push no ramo main.
Azure Cosmos DB	Base de dados NoSQL principal com 6 containers: jogadores, utilizadores, equipas, ligas, pontuações e eventos_jornada.
Docker Container	Backend Flask containerizado com Dockerfile baseado em python:3.11-slim, usando Gunicorn como servidor WSGI.
Azure Functions	Três funções serverless: GerarPontuacoes, AtualizarPrecos e EventosEspeciais, acionadas por timer trigger semanal.
Azure BLOB Storage	Armazenamento dos 18 logos dos clubes da Liga Portugal e imagens de perfil dos utilizadores.
Azure Container Registry	Repositório privado para a imagem Docker do backend (onzeperfeito-backend:1.0).
ARM / Bicep (IaC)	Template Bicep com 238 linhas que permite recriar toda a infraestrutura Azure com um único comando.

4. Arquitetura da Solução

A arquitetura do OnzePerfeito segue o modelo de aplicação web em **três camadas**: apresentação, lógica de negócio e dados.

Camada	Tecnologia	Serviço Azure
Apresentação	HTML5, CSS3, JavaScript	Azure App Service (static/)
Lógica de Negócio	Python 3.11 + Flask + Gunicorn	Azure App Service (Docker, B1 Basic)
Dados	NoSQL (Cosmos DB for NoSQL)	Azure Cosmos DB Serverless
Ficheiros Estáticos	PNG (logos clubes)	Azure BLOB Storage (LRS)
Automação	JavaScript (Node.js 22 LTS)	Azure Functions (Consumption)
CI/CD	GitHub Actions	App Service Deploy Workflow

O fluxo típico de interação é o seguinte: o utilizador acede à aplicação através do browser; o frontend envia pedidos HTTP à API Flask, que por sua vez consulta ou atualiza o CosmosDB. As Azure Functions são acionadas por temporizador (*timer trigger*) às sextas-feiras, gerando as pontuações e atualizando os preços dos jogadores automaticamente, sem qualquer intervenção manual.

5. Planificação e Divisão de Tarefas

O desenvolvimento do projeto foi organizado em três fases, correspondentes às três avaliações definidas no enunciado, com distribuição de tarefas equilibrada entre os membros do grupo.

Tarefa	Responsável	Fev	Mar	Abr	Mai
Relatório #1 — Ideia e planificação	Todos	✓			
Configuração do repositório GitHub	Rodrigo	✓			
Template Bicep (infraestrutura Azure)	Rodrigo		✓		
Script de seed (jogadores CosmosDB)	Kyrylo		✓		
Backend Flask — API REST	João Pedro		✓	✓	
Integração CosmosDB com backend	Kyrylo		✓	✓	
Azure Functions (pontuações e preços)	Rodrigo			✓	
Relatório #2 — Estado intermédio	Todos			✓	
Frontend HTML/CSS/JS	João Pedro			✓	✓
BLOB Storage (imagens)	Kyrylo			✓	✓
Deploy em produção	Todos				✓
Melhorias (Vista Tática, Ligas)	Todos				✓
Testes de integração	Todos				✓
Estudo de custos e alternativas	Todos				✓
Relatório #3 — Versão final	Todos				✓

5.1. Divisão de Responsabilidades

João Pedro de Sousa Pereira (nº 20221689) — Backend Flask (API REST) e Frontend (HTML/CSS/JS).

Kyrylo Sakhnenko (nº 20221710) — Integração CosmosDB, script de seed de jogadores e configuração do BLOB Storage.

Rodrigo Coimbra Figueiredo (nº 20221713) — Repositório GitHub, template Bicep de infraestrutura e Azure Functions.

Relatórios, testes e apresentações foram realizadas em conjunto pelos três elementos.

6. Infraestrutura Base no Azure

Todos os recursos Azure foram criados na região **France Central** (a opção West Europe apresentou erros durante a criação), agrupados no resource group **onzeperfeito-rg**, com subscrição Azure for Students.

6.1. Resource Group e Azure Cosmos DB

Parâmetro	Valor
Nome	onzeperfeito-rg
Região	France Central
Subscrição	Azure for Students

O Azure Cosmos DB foi criado em modo **Serverless**, ideal para o padrão de uso irregular do OnzePerfeito (picos às sextas-feiras). Foram criados **6 containers NoSQL**:

Container	Partition Key	Finalidade
jogadores	/clube	Jogadores da Liga Portugal
utilizadores	/email	Utilizadores registados
equipas	/utilizador_id	Equipas de cada utilizador
ligas	/tipo	Ligas globais e privadas
pontuações	/jornada	Pontuações por jornada
eventos_jornada	/jornada	Eventos Especiais por Jornada

Parâmetro	Valor
Nome da conta	onzeperfeito-cosmos
API	Azure Cosmos DB for NoSQL
Modo	Serverless
Região	France Central

6.2. Azure BLOB Storage

Foi criada uma conta de armazenamento para guardar os logos dos clubes e imagens de perfil, com um container público *imagens* com acesso Blob (leitura pública sem autenticação).

Parâmetro	Valor
Nome	onzeperfeitostorage
Redundância	Locally-redundant storage (LRS)
Container	imagens (acesso público Blob)
Região	France Central

7. GitHub e Azure App Service — CI/CD

Foi criado o repositório público **onzeperfeito** na conta KyryloSakhnenko21, com os ficheiros base do backend (app.py, requirements.txt, Dockerfile) e o workflow de GitHub Actions para deploy automático a cada push no ramo *main*.

Parâmetro	Valor
Repositório	github.com/KyryloSakhnenko21/onzeperfeito
Visibilidade	Público
Ramo principal	main

7.1. Azure App Service

O App Service foi inicialmente criado no plano gratuito F1 com Python 3.11. A integração com GitHub Actions permite deploy automático a cada push. Durante a configuração surgiram dois erros resolvidos iterativamente: o *requirements.txt* estava na pasta *backend/* e o GitHub não tinha as credenciais Azure — ambos corrigidos ajustando o workflow e adicionando o Publish Profile como secret.

Posteriormente, o plano foi atualizado para **B1 Basic** após esgotamento da quota de CPU do F1 (60 min/dia). O plano B1 não tem limite de CPU e custa aproximadamente 13€/mês, coberto pelos créditos Azure for Students.

Parâmetro	Valor
Nome	onzeperfeito-app
URL	onzeperfeito-app.azurewebsites.net
Runtime	Python 3.11
Plano (final)	B1 Basic Linux
Região	France Central

8. Backend Flask e Docker

O backend foi desenvolvido em **Python com Flask**, seguindo uma arquitetura modular em **Blueprints**. Foram implementadas 13 rotas REST organizadas por área funcional.

8.1. Arquitetura e Rotas REST

Blueprint	Rotas Principais
auth_bp	POST /auth/register POST /auth/login GET /auth/utilizador/
jogadores_bp	GET /jogadores/ GET /jogadores/
equipas_bp	GET /equipas/ POST /equipas/
ligas_bp	GET /ligas/ POST /ligas/criar POST /ligas/entrar
jornadas_bp	GET /jornadas/pontuacoes GET /jornadas/evento

8.2. Containerização com Docker e Azure Container Registry

A aplicação foi containerizada com um Dockerfile baseado em **python:3.11-slim**, usando **Gunicorn** como servidor WSGI de produção. A imagem foi testada localmente antes de ser publicada no Azure Container Registry.

Parâmetro	Valor
Nome	onzeperfeitoRegistry
Login server	onzeperfeitoregistry.azurecr.io
Imagem	onzeperfeito-backend:1.0
Plano	Basic

O workflow GitHub Actions foi simplificado para enviar apenas o código fonte (sem *venv* local), deixando o sistema Oryx do Azure instalar as dependências nativamente — solução que eliminou incompatibilidades de GLIBC entre o Ubuntu do GitHub Actions e o Linux do App Service.

9. Azure Functions

Foram implementadas três Azure Functions para automatizar as operações periódicas da plataforma. O código JavaScript foi desenvolvido com auxílio da ferramenta de IA Claude. A Function App foi criada em **North Europe** devido a restrições do resource group para Linux Consumption em France Central.

Parâmetro	Valor
Nome	onzeperfeito-functions
Runtime	Node.js 22 LTS
Região	North Europe
Plano	Consumption (Serverless)

9.1. GerarPontuacoes

Corre automaticamente todas as sextas-feiras às 18h00 (CRON: 0 0 18 * * 5). Gera pontuações aleatórias para todos os jogadores dentro dos intervalos definidos por posição e regista-as no CosmosDB.

9.2. AtualizarPrecos

Corre às 18h30 das sextas (CRON: 0 30 18 * * 5), 30 minutos após a geração de pontuações. Calcula a média de cada jogador e atualiza o preço: **+10%** se média > 8, **-10%** se média < 4, com limites de 4.0 a 30.0.

9.3. EventosEspeciais — Funcionalidade Inovadora

Corre às 12h00 das sextas (CRON: 0 0 12 * * 5), 6 horas antes das pontuações, revelando o evento especial da jornada. Sorteia aleatoriamente um de 6 eventos possíveis que alteram as regras de pontuação dessa jornada:

Evento	Posição Afetada	Multiplicador
Semana dos Avançados	Avançado	× 2.0
Semana dos Defesas	Defesa	× 2.0
Semana dos Médios	Médio	× 2.0
Semana dos Guarda-Redes	Guarda-redes	× 2.0
Jornada de Ouro	Todos	× 1.5
Jornada Surpresa	Posição aleatória	× 3.0

10. Script de Seed dos Jogadores

Foi criado e executado um script Python para popular o CosmosDB com **487 jogadores reais** da Liga Portugal Betclic 2025-26, distribuídos pelos 18 clubes da competição. Os dados foram obtidos a partir de sites oficiais dos clubes, Sofascore, FotMob, Tribuna e Wikipedia.

Parâmetro	Valor
Linguagem	Python 3.11 com SDK azure-cosmos
Ficheiro	onzeperfeito/seed/seed_jogadores.py
Total inserido	487 jogadores em 18 clubes

Cada jogador foi inserido com os campos: **id** (UUID), **nome**, **clube** (partition key), **posição**, **preço inicial** e **media_pontuacao** (inicializada a 0.0). Os preços foram definidos manualmente numa escala de 4.0 a 18.0, refletindo a qualidade de cada jogador.

Clube	Jogadores	Clube	Jogadores
Sporting CP	28	CF Estrela da Amadora	28
FC Porto	30	FC Alverca	33
SL Benfica	30	Gil Vicente FC	31
SC Braga	30	GD Estoril Praia	29
CD Santa Clara	30	FC Arouca	27
CD Nacional	25	Casa Pia AC	23
Vitória SC	23	CD Tondela	26
FC Famalicão	23	AVS Futebol SAD	28
Moreirense FC	22	Rio Ave FC	21

11. Template Bicep — Infrastructure as Code

Foi criado manualmente um template **Bicep** que permite recriar toda a infraestrutura Azure do projeto com um único comando. O Bicep é uma linguagem de IaC desenvolvida pela Microsoft como alternativa mais legível ao formato ARM JSON. A criação manual (em vez de exportar pelo portal) foi uma decisão deliberada para demonstrar compreensão genuína do conceito de IaC.

Parâmetro	Valor
Ficheiro	infra/main.bicep
Repositório	github.com/KyryloSakhnenko21/onzeperfeito
Linhas	238 linhas (7.25 KB)
Recursos definidos	CosmosDB, BLOB Storage, Container Registry, App Service, Function App
Parâmetros	5 parâmetros configuráveis com valores por defeito
Outputs	cosmosEndpoint, appServiceUrl, registryLoginServer, storageAccountName

O template usa loops declarativos para criar os 6 containers do CosmosDB de forma breve, e define dois App Service Plans distintos — B1 Basic para o App Service em France Central e Y1 Dynamic para as Functions em North Europe. Para recriar toda a infraestrutura basta executar:

```
az deployment group create --resource-group onzeperfeito-rg --template-file infra/main.bicep
```

12. Frontend — Desenvolvimento da Interface

O frontend foi desenvolvido com tecnologias web puras — **HTML5, CSS3 e JavaScript** — sem recurso a frameworks como React ou Vue.js. Esta decisão mantém a consistência com a stack tecnológica definida inicialmente, evitando complexidade adicional no deploy e mantendo a legibilidade do código para fins académicos.

12.1. Arquitetura e Ficheiros de Base

O frontend está organizado em ficheiros partilhados e páginas individuais:

Ficheiro	Tipo	Descrição
style.css	CSS	Estilos globais, sistema de variáveis CSS, componentes UI
auth.js	JavaScript	Módulo de gestão de sessão JWT do lado do cliente
api.js	JavaScript	Wrapper centralizado para chamadas ao backend Flask
navbar.js	JavaScript	Componente de navegação injetado dinamicamente
index.html	HTML	Página de login e registo
dashboard.html	HTML	Dashboard principal pós-login
mercado.html	HTML	Mercado de jogadores com filtros e paginação
equipa.html	HTML	Gestão da equipa do utilizador (vista lista + vista tática)
ligas.html	HTML	Gestão de ligas globais e privadas
classificacoes.html	HTML	Rankings e pontuações por jornada

Tema Visual

O tema visual foi inspirado na Liga Portugal, combinando verde escuro com dourado numa estética desportiva e profissional, com todas as cores e fontes definidas como variáveis CSS.

12.2. Módulos auth.js e api.js

auth.js — Gestão de Autenticação JWT

O módulo encapsula toda a lógica de sessão do lado do cliente. A autenticação baseia-se em JWT emitidos pelo backend Flask. O token e os dados do utilizador são guardados no *localStorage* do browser em três chaves:

Chave localStorage	Conteúdo
op_token	Token JWT — enviado no cabeçalho Authorization
op_nome	Nome do utilizador — exibido na navbar

Chave localStorage	Conteúdo
op_id	ID do utilizador (UUID) — usado nas rotas

O módulo verifica automaticamente se o token expirou fazendo o decode do payload JWT (Base64) e comparando o campo exp com o timestamp atual, sem necessidade de contactar o servidor. Se o token expirar, a sessão é terminada e o utilizador redirecionado para login.

api.js — Wrapper de Comunicação

Centraliza todas as chamadas HTTP ao backend, injetando automaticamente o token JWT no cabeçalho Authorization. Se o servidor devolver um 401, a sessão é terminada imediatamente.

Namespace	Método	Rota Backend
API.auth	registar(), login()	POST /auth/register, /auth/login
API.jogadores	listar(), obter()	GET /jogadores/, /jogadores/
API.equipas	obter(), guardar()	GET, POST /equipas/
API.ligas	listar(), criar(), entrar()	GET /ligas/, POST /ligas/criar, /entrar
API.jornadas	pontuacoes(), evento()	GET /jornadas/pontuacoes, /evento

12.3. Páginas HTML Desenvolvidas

index.html — Login e Registo

Ponto de entrada da aplicação (única página sem autenticação). Apresenta formulário com dois separadores (tabs): Entrar e Registar. Inclui validação client-side, toggle de password, indicador de carregamento durante pedidos HTTP, e redirecionamento automático para o dashboard se já autenticado.

dashboard.html — Painel Principal

Primeira página após o login. Apresenta: saldo disponível (100M€  custo da equipa), contagem de jogadores, pontuação total calculada a partir do CosmosDB, e evento especial da jornada com destaque visual.

mercado.html — Mercado de Jogadores

Lista os 487 jogadores com filtros por nome (em tempo real), posição, clube e ordenação. Paginação a 20 jogadores por página. Antes de adicionar um jogador, verifica: limite de 15 jogadores, saldo disponível, e se o jogador já está na equipa.

equipa.html — Gestão da Equipa

Apresenta os 15 jogadores com separação entre titulares e suplentes. Inclui barra de progresso de vagas preenchidas, distribuição por posição, e botão de remoção. Expandida na Fase de Melhorias com Vista Tática (ver Secção 14.1).

ligas.html — Gestão de Ligas

Liga global e ligas privadas. Permite criar ligas (código de 6 caracteres gerado pelo backend) e entrar com código. Cópia do código para a área de transferência com um clique. Expandida na Fase de Melhorias com tabela de classificação (ver Secção 14.2).

classificacoes.html — Rankings

Rankings por jornada com pódio visual para os três primeiros e tabela completa. O utilizador autenticado é destacado na tabela. Navegação entre jornadas 1-34 por botões.

13. Integração BLOB Storage e Deploy em Produção

13.1. Logos dos Clubes e Configuração de CORS

Os logos dos 18 clubes da Liga Portugal Betclic foram descarregados manualmente e carregados para o container *imagens* do Azure BLOB Storage. O acesso público Blob permite que os logos sejam servidos diretamente através de URL sem autenticação.

Para que o browser possa carregar os logos a partir da aplicação (domínio diferente), foram configuradas as regras de CORS no serviço Blob:

Parâmetro CORS	Valor	Justificação
Allowed origins	*	Acesso de qualquer domínio — adequado para projeto académico
Allowed methods	GET, OPTIONS	Leitura de imagens; OPTIONS para preflight requests
Allowed headers	*	Aceita qualquer cabeçalho no pedido
Exposed headers	*	Expõe todos os cabeçalhos na resposta
Max age	86400	Cache da política CORS por 24 horas

13.2. Fotos dos Jogadores — Wikipedia API com Fallback

Dado o elevado número de jogadores (487), não seria viável fazer upload manual de fotos. A solução adotada combina duas abordagens: tentativa de carregar a foto real via **API pública da Wikipedia**, com fallback automático para um **avatar gerado com as iniciais** do jogador (cor baseada no clube) caso a foto não esteja disponível. O avatar com iniciais é exibido imediatamente, e a foto da Wikipedia substitui-o em background se disponível (timeout de 4 segundos).

Aspeto	Logos dos Clubes (BLOB)	Fotos dos Jogadores (Wikipedia)
N.º de imagens	18 (fixo)	487 (variável por época)
Origem	Upload manual pelo portal	API pública da Wikipedia
Fiabilidade	100% — imagens controladas	Variável — depende da Wikipedia
Custo de manutenção	Baixo — atualizar por época	Zero — automático
Fallback	Iniciais do clube	Avatar com iniciais e cor do clube

13.3. Problemas de Deploy e Soluções

O processo de deploy revelou múltiplos problemas técnicos, documentados abaixo como registo das dificuldades reais de deploy em ambiente cloud:

Problema: Quota F1 Excedida

Erro	O plano gratuito F1 tem limite de 60 min CPU/dia. Após múltiplos testes, a quota foi esgotada.
Solução	Upgrade do plano F1 para B1 Basic (~13€/mês, coberto pelos créditos Azure for Students).

Problema: Chave do CosmosDB Exposta no GitHub

Erro	GitHub detetou a chave primária do CosmosDB hardcoded no seed_jogadores.py e bloqueou o push.
Solução	Remoção do ficheiro do histórico com git filter-repo, regeneração da chave e uso de variáveis de ambiente.

Problema: Frontend Não Servido pelo Flask

Erro	Deploy devolvia 404 na rota /app. O Azure Oryx usava cache antiga (antenv.tar.gz).
Solução	Eliminação manual dos ficheiros de cache via SSH do Kudu e restart do App Service.

Problema: Incompatibilidade de GLIBC

Erro	O venv criado no GitHub Actions (Ubuntu 22.04, GLIBC 2.35) era incompatível com o Linux do Azure.
Solução	Workflow simplificado: apenas código fonte no zip, sem venv. Oryx instala dependências nativamente.

Problema: App Service com URL de Slot Antigo

Erro	Após recriar o App Service com Bicep, o URL antigo deixou de funcionar.
Solução	Download do novo Publish Profile pelo portal e atualização do secret no GitHub.

Problema: Variáveis de Ambiente em Falta

Erro	Após recriar o App Service, COSMOS_KEY e JWT_SECRET não estavam configuradas.
Solução	Adição manual das variáveis de ambiente no portal Azure (Environment variables → App settings).

13.4. Arquitetura Final em Produção

Componente	Configuração	URL / Localização
Frontend	Servido pelo Flask via static/	onzeperfeito-app.azurewebsites.net

Backend Flask	Python 3.11, Gunicorn, B1 Basic	onzeperfeito-app.azurewebsites.net/auth...
Base de dados	CosmosDB Serverless, France Central	onzeperfeito-cosmos.documents.azure.com
Logos dos clubes	BLOB Storage, acesso público Blob	onzeperfeitostorage.blob.core.windows.net
Fotos jogadores	Wikipedia API + avatar fallback	en.wikipedia.org/w/api.php
CI/CD	GitHub Actions  App Service	github.com/KyryloSakhnenko21/onzeperfeito
Azure Functions	Node.js 22, Consumption, N. Europe	onzeperfeito-functions (3 funções)

14. Melhorias — Fase Final

Na fase de melhorias do projeto, foram implementadas duas evoluções significativas à experiência de utilizador da plataforma: a Vista Tática com formações interativas no módulo de equipa, e a reformulação do módulo de ligas com tabela de classificação.

14.1. Vista Tática com Formações — equipa.html

Até esta fase, a página *A Minha Equipa* apresentava os jogadores exclusivamente em formato de tabela. A melhoria transforma a gestão da equipa numa experiência visual semelhante à de plataformas de fantasy football profissionais, com um campo de futebol interativo, seleção de formação e validação de posições em tempo real.

Formações Táticas Disponíveis

Formação	GR	DEF	MED	AV	Descrição
4-3-3	1	4	3	3	Clássica, ofensiva, 3 avançados
4-4-2	1	4	4	2	Equilibrada, dois avançados
3-5-2	1	3	5	2	Controlo a meio-campo
4-2-3-1	1	4	5*	1	Médio defensivo + atacante a só
5-3-2	1	5	3	2	Defensiva, 5 defesas
3-4-3	1	3	4	3	Ultra-ofensiva, 3 avançados

Funcionalidades da Vista Tática

- Campo de futebol renderizado em CSS com overlay SVG (linhas, áreas, círculo central, arcos)
- Slots dos jogadores posicionados em coordenadas percentuais absolutas para cada formação
- Validação de posições: cada slot só aceita jogadores da posição correspondente (dragover + drop)
- Drag & drop nativo do HTML5 com feedback visual imediato no dragover
- Clique simples na bancada — coloca automaticamente no primeiro slot vazio compatível
- Auto-preenchimento ao carregar a equipa e ao mudar de formação
- Tooltip ao clicar num slot ocupado (nome, posição, clube, média) com opção de remover
- Bancada filtrável por posição (Todos / GR / DEF / MED / AV)
- Suplentes dinâmicos: todos os jogadores não em campo aparecem como suplentes
- Vista Lista original mantida como alternativa via tab

Correções Implementadas

Problema	Causa	Solução
Suplentes bloqueados na bancada	Função excluía índices 11-14 assumindo-os sempre suplentes	Bancada inclui todos os 15 jogadores; titular/suplente depende do slot do campo
Auto-preenchimento limitado	equipaAtual.slice(0, 11) ignorava os 4 últimos jogadores	Função itera sobre equipaAtual completo

14.2. Módulo de Ligas — `ligas.html`

Antes desta melhoria, a página de ligas não disponibilizava qualquer forma de aceder ao interior de uma liga para consultar os seus membros ou a sua classificação. Foram identificadas e resolvidas as seguintes necessidades:

Problema identificado	Impacto para o utilizador
Impossibilidade de entrar no detalhe de uma liga	Não era possível ver quem estava inscrito
Ausência de tabela de classificação por liga	Sem ranking dos membros por pontuação
Erro crítico (API_BASE redeclarado)	Página ficava em spinner infinito
Nomes dos membros não visíveis (apenas IDs)	Identificação impossível dos participantes

Correção do Erro Crítico de JavaScript

O browser reportava `Uncaught SyntaxError: Identifier 'API_BASE' has already been declared`. A constante já estava definida em `api.js` e era redeclarada com `var` no script inline de `ligas.html`. A resolução passou pela remoção da linha duplicada, usando a variável global já disponível.

Nova Vista de Detalhe de Liga

Foi criada uma segunda vista dentro da mesma página — acessível ao clicar em qualquer card de liga. A navegação entre as duas vistas é feita por manipulação de CSS (`display: none/block`), sem recarregamentos de página. Foi implementado um dicionário **ligasCache** em memória para passagem segura de dados entre vistas, evitando serialização de JSON em atributos HTML (que causava erros com caracteres especiais).

Tabela de Classificação com Pontuações Reais

A vista de detalhe inclui uma tabela de classificação com todos os membros da liga ordenados pela pontuação acumulada. O processo envolve três pedidos paralelos à API via `Promise.all`:

Pedido	Endpoint	Finalidade
Nomes dos membros	<code>GET /auth/utilizador/{id}</code>	Nome de cada membro da liga
Equipas	<code>GET /equipas/?utilizador_id={id}</code>	Jogadores de cada equipa
Pontuações	<code>GET /jornadas/pontuacoes?jornada={n}</code>	Pontos de cada jogador por jornada

A tabela apresenta medalhas para os três primeiros lugares e destaque visual para a linha do utilizador autenticado (badge "Tu").

Novo Endpoint — `GET /auth/utilizador/<id>`

Para suportar a apresentação dos nomes reais, foi adicionado um novo endpoint ao backend Flask (`auth.py`). A query ao CosmosDB usa projeção (`SELECT c.id, c.nome`) para garantir que apenas os campos necessários são transmitidos, minimizando o consumo de RUs e a exposição de dados. Todos os pedidos usam o token JWT do utilizador autenticado, injetado pelo módulo `api.js`.

Ficheiro	Alteração
ligas.html	Correção SyntaxError (API_BASE redeclarado)
ligas.html	Substituição de onclick inline por addEventListener
ligas.html	Implementação do objeto ligasCache para passagem segura de dados
ligas.html	Vista de detalhe de liga (navegação sem recarregamento)
ligas.html	Tabela de classificação com nomes reais, pontos e medalhas
ligas.html	Destaque visual da linha do utilizador autenticado (badge 'Tu')
auth.py	Novo endpoint GET /auth/utilizador/ para obter nome por ID

15. Testes de Integração

Foram realizados testes de integração sobre a aplicação em produção, com o objetivo de validar o correto funcionamento dos principais fluxos da plataforma. Os testes foram executados manualmente através do browser Opera, recorrendo às DevTools (separador Network) para inspeção dos pedidos HTTP.

Parâmetro	Valor
Ferramenta	Opera Browser + DevTools (separador Network)
Ambiente	Produção — onzeperfeito-app.azurewebsites.net
Tipo de testes	Testes de integração manuais
Critério de sucesso	Código HTTP esperado + comportamento correto da interface
Data de execução	Maio de 2026

15.1. Resultados dos Testes

Nº	Teste	Resultado
1	Registo com dados válidos (status 201 + mensagem de sucesso)	✓ Passou
2	Registo com email já existente (status 409 + mensagem de erro)	✓ Passou
3	Registo com campos em falta (validação HTML5 + mensagem no ecrã)	✓ Passou
4	Login com credenciais corretas (status 200 + redireccionamento)	✓ Passou
5	Login com password incorreta (status 401 + mensagem de erro)	✓ Passou
6	Acesso a rota protegida sem autenticação (redirect para login)	✓ Passou
7	Listagem de jogadores no Mercado (status 200 + 487 jogadores)	✓ Passou
8	Adição de jogador à equipa (status 200 + persistência no CosmosDB)	✓ Passou
9	Criação de liga privada com código de 6 caracteres	✓ Passou
10	Entrar em liga com código inválido (erro de validação frontend)	✓ Passou

15.2. Teste End-to-End — Fluxo Completo

Foi executado um teste de integração end-to-end que simula uma sessão completa de um utilizador real, validando a integração entre todos os componentes da infraestrutura Azure:

Passo	Ação	Resultado Esperado	Estado
1	Registar novo utilizador	Status 201 + redireccionamento	✓ OK
2	Login com as credenciais	Status 200 + token JWT + dashboard	✓ OK
3	Navegar para o Mercado	Lista de 487 jogadores carregada	✓ OK
4	Adicionar 2 jogadores	Status 200 + saldo atualizado	✓ OK
5	Navegar para A Minha Equipa	Os 2 jogadores visíveis na equipa	✓ OK

6	Navegar para o Dashboard	Saldo reduzido conforme custo dos jogadores	
---	--------------------------	---	--

16. Estudo de Custos e Alternativas Cloud

16.1. Estimativa de Custos Mensais em Produção

Os custos mensais foram calculados com base nos preços públicos da Azure Pricing Calculator para as regiões France Central e North Europe. Câmbio utilizado: 1 USD = 0.93 EUR (Maio de 2026).

App Service — Plano B1 Basic (Linux, France Central)

O App Service aloja o backend Flask que serve simultaneamente o frontend estático. O plano B1 oferece 1 vCPU, 1.75 GB RAM e 10 GB de armazenamento, sendo o mínimo que suporta domínios personalizados, SSL e deployment slots.

Subtotal App Service: **\$13.14/mês (~€12.22/mês)**

Azure Cosmos DB — Serverless (France Central)

O modelo Serverless cobra por Request Units (RUs) efetivamente consumidas, sem custos fixos. É ideal para o OnzePerfeito dado o tráfego irregular (picos às sextas-feiras).

Operação	Freq. Estimada	RUs/Op.	RUs/mês
Login/Registo	500 ops/mês	5 RU	2.500 RU
Leitura de jogadores	8.000 ops/mês	3 RU	24.000 RU
Escrita/atualização de equipa	1.000 ops/mês	10 RU	10.000 RU
Leitura de pontuações	5.000 ops/mês	5 RU	25.000 RU
Azure Functions (487 players)	4 exec. × 487	10 RU	19.480 RU
Ligas e eventos	500 ops/mês	8 RU	4.400 RU
TOTAL ESTIMADO			~85.380 RU/mês

Subtotal Cosmos DB: **~\$0.10/mês (~€0.09/mês)**

Azure Functions, BLOB Storage e Container Registry

Serviço	Detalhe	Custo/mês
Azure Functions	12 execuções/mês — 100% coberto pelo Free Tier	\$0.00
BLOB Storage LRS Hot	~5 MB logs + ~50.000 GET/mês	~\$0.02
Container Registry Basic	onzeperfeitoregistry.azurecr.io, 10 GB incluídos	\$5.00

Resumo Total — Custo Mensal Estimado

Serviço Azure	Região	SKU / Plano	USD/mês	EUR/mês
App Service	France Central	B1 Basic Linux	\$13.14	€12.22

Cosmos DB for NoSQL	France Central	Serverless	\$0.10	€0.09
BLOB Storage	France Central	LRS Hot	\$0.02	€0.02
Azure Functions	North Europe	Consumption (Y1)	\$0.00	€0.00
Container Registry	France Central	Basic	\$5.00	€4.65
Total Mensal			\$18.26	€16.98

16.2. Alternativas Cloud Europeias

Foi realizado um estudo comparativo entre o Azure e três alternativas cloud europeias, considerando custo, conformidade GDPR, facilidade de uso, equivalência de serviços e soberania de dados.

OVHcloud (França) — "Le Cloud Souverain Européen"

A OVHcloud é a maior empresa de cloud europeia por quota de mercado, com sede em Roubaix, França. Destaca-se pela certificação SecNumCloud (ANSSI) — a mais rigorosa na Europa — garantindo soberania de dados 100% em território francês/europeu, sem exposição ao CLOUD Act americano. O ecossistema PaaS é menos maduro e o custo estimado para o perfil do OnzePerfeito seria **~€25/mês** (mais caro que o Azure, pois não existe equivalente serverless económico ao Cosmos DB).

Scaleway (França) — Cloud Developer-Friendly

Subsidiária do grupo Iliad, com sede em Paris. Conhecida pela transparência de preços, API bem documentada e forte presença na comunidade open-source europeia. O Container Registry é **gratuito**, o modelo Serverless Containers seria um excelente substituto ao App Service, e o custo estimado seria **~€13/mês** — inferior ao Azure com soberania europeia garantida.

Hetzner Cloud (Alemanha) — A Opção Mais Económica

Empresa alemã fundada em 1997, amplamente reconhecida como uma das opções mais económicas do mercado europeu. Foca-se em IaaS (VMs, Storage) com PaaS limitado — sem Serverless Functions nativo nem Container Registry. O custo estimado seria **~€7/mês** (a opção mais económica), mas requereria substituir as Azure Functions por cron jobs num servidor e mais configuração manual.

Tabela Comparativa Final

Critério	Azure	OVHcloud	Scaleway	Hetzner
Custo mensal estimado	€16.98	~€25	~€13	~€7
Economia/Preço (1-5)	■■■	■■	■■■■	■■■■■
Conformidade GDPR (1-5)	■■■■	■■■■■	■■■■■	■■■■■
Soberania de Dados (1-5)	■■ (CLOUD Act)	■■■■■	■■■■■	■■■■■
Facilidade de Uso / PaaS (1-5)	■■■■■	■■■	■■■■	■■
Serverless (1-5)	■■■■■	■■■	■■■■	■ (n/a)
Integração CI/CD (1-5)	■■■■■	■■■	■■■■	■■

Documentação/Suporte (1-5)	■■■■■	■■■	■■■■	■■■
Programa Estudantes (1-5)	■■■■■ (\$100)	■■	■■■	■
Container Registry (1-5)	■■■■	■■■	■■■■■ (grátis)	■ (n/a)
PONTUAÇÃO TOTAL (/55)	43/55	34/55	44/55	30/55

Conclusão: Para o contexto académico atual, o Azure é a escolha correta pelo programa de estudantes, integração GitHub e Cosmos DB Serverless económico. Para um cenário de produção real com requisitos GDPR reforçados, a **Scaleway** seria a alternativa europeia mais recomendada, com custo inferior e soberania de dados garantida.

17. Conclusão

A plataforma **OnzePerfeito** foi desenvolvida com sucesso ao longo do semestre, concretizando todos os objetivos definidos no enunciado e indo além deles com funcionalidades inovadoras como os Eventos Especiais por Jornada e a Vista Tática com drag & drop.

O projeto demonstrou, na prática, o valor e os desafios de uma arquitetura cloud moderna: a combinação de Azure App Service, Cosmos DB Serverless, Azure Functions e BLOB Storage provou ser tecnicamente adequada e economicamente viável (~€17/mês), com deploy automatizado via GitHub Actions e infraestrutura reprodutível via Bicep.

18. Referências

- [1] Fantasy Premier League (2024). *How to Play*. Premier League.
Disponível em: <https://fantasy.premierleague.com/help/rules>
- [2] Microsoft (2026). *Azure App Service documentation*. Microsoft Learn.
Disponível em: <https://learn.microsoft.com/en-us/azure/app-service/>
- [3] Microsoft (2026). *Introduction to Azure Cosmos DB*. Microsoft Learn.
Disponível em: <https://learn.microsoft.com/en-us/azure/cosmos-db/introduction>
- [4] Microsoft (2026). *Azure Functions documentation*. Microsoft Learn.
Disponível em: <https://learn.microsoft.com/en-us/azure/azure-functions/>
- [5] Docker Inc. (2024). *Docker Documentation*. Docker. Disponível em: <https://docs.docker.com>
- [6] Microsoft (2026). *Azure Container Registry documentation*. Microsoft Learn.
Disponível em: <https://learn.microsoft.com/en-us/azure/container-registry/>
- [7] Microsoft (2026). *What is Bicep?*. Microsoft Learn. Disponível em:
<https://learn.microsoft.com/en-us/azure/azure-resource-manager/bicep/overview>
- [8] GitHub (2026). *GitHub Actions documentation*. GitHub Docs. Disponível em:
<https://docs.github.com/en/actions>
- [9] Sofascore (2026). *Liga Portugal Betclíc 2025/26*. Disponível em:
<https://www.sofascore.com/tournament/football/portugal/liga-portugal/238>
- [10] FotMob (2026). *Liga Portugal 2025/26*. Disponível em:
<https://www.fotmob.com/leagues/87/overview/liga-portugal>
- [11] Tribuna Expresso (2026). *Liga Portugal Betclíc — Notícias e plantéis 2025/26*.
Disponível em: <https://tribuna.expresso.pt/liga-portugal>
- [12] Wikipedia (2026). *Liga Portugal 2025-26*. Disponível em:
https://pt.wikipedia.org/wiki/Liga_Portugal_2025-26
- [13] Liga Portugal (2026). *Liga Portugal Betclíc — Época 2025/26*. Disponível em:
<https://www.ligaportugal.pt>
- [14] OVHcloud (2026). *OVHcloud Pricing*. Disponível em: <https://www.ovhcloud.com/pt/>
- [15] Scaleway (2026). *Scaleway Pricing*. Disponível em: <https://www.scaleway.com/en/pricing/>
- [16] Hetzner Online (2026). *Hetzner Cloud Pricing*. Disponível em: <https://www.hetzner.com/cloud>
- [17] Microsoft (2026). *Azure Pricing Calculator*. Disponível em: <https://azure.microsoft.com/en-us/pricing/calculator/>